

Índice

Stockly — Sistema de gestión y reservas de almacén	3
Portada	3
Resumen	3
Abstract	4
Palabras clave	4
Índice	4
1. Introducción	7
1.1 Contexto del proyecto	7
1.2 Objetivos del proyecto	8
1.3 Motivación	8
1.4 Tecnologías utilizadas	8
2. Análisis y diseño del sistema	9
2.1 Descripción general del proyecto	9
2.2 Roles de usuario	9
2.3 Requisitos funcionales	9
2.4 Requisitos no funcionales	10
2.5 Casos de uso	11
2.6 Diseño de interfaces (Figma)	11
2.7 Arquitectura general del sistema	12
3. Diseño de la base de datos	13
3.1 Modelo entidad-relación	13
3.2 Diseño lógico de tablas	13
3.3 Relaciones y claves	13
3.4 Scripts SQL	13
3.5 Datos de prueba (seed)	14
4. Desarrollo del backend	14
4.1 Arquitectura del backend	14
4.2 Configuración inicial del proyecto	14
4.3 Conexión a la base de datos	15
4.4 Desarrollo de la API REST	15
4.5 CRUD de entidades	15
4.6 Validaciones de datos	16
4.7 Manejo global de errores	16
4.8 Autenticación y autorización	16
4.9 Swagger / OpenAPI	17
4.10 Pruebas de endpoints	17
5. Desarrollo de la aplicación web	18
5.1 Diseño inicial en Figma	18
5.2 Estructura del frontend	18
5.3 Integración con la API REST	18
5.4 Gestión de sesiones y login	19
5.5 CRUD de datos	19

5.6 Funcionalidades según rol	19
5.7 Manejo de errores	19
5.8 Capturas de la aplicación	19
6. Desarrollo de la aplicación móvil	20
6.1 Diseño inicial en Figma	21
6.2 Navegación entre pantallas	21
6.3 Conexión con la API REST	21
6.4 Persistencia local	22
6.5 Gestión de sesiones	22
6.6 Funcionalidades futuras	22
6.7 Capturas de la aplicación	22
7. Despliegue e infraestructura	23
7.1 Variables de entorno	23
7.2 Despliegue del backend	24
7.3 Base de datos remota	24
7.4 Hosting y servicios cloud	24
7.5 Git y GitHub	25
7.6 GitHub Actions / CI/CD	25
7.7 Acceso público al sistema	25
8. Seguridad	26
8.1 Autenticación	26
8.2 Autorización	26
8.3 Protección de datos	26
8.4 Buenas prácticas aplicadas	26
9. Inteligencia Artificial aplicada al proyecto	27
9.1 Herramientas utilizadas	27
9.2 Uso realizado	27
9.3 Validación y revisión	27
10. Pruebas y validación	28
10.1 Pruebas funcionales	28
10.2 Casos de prueba	28
10.3 Resultados obtenidos	29
10.4 Errores encontrados y soluciones	29
11. Conclusiones	29
11.1 Resultados obtenidos	29
11.2 Problemas encontrados	30
11.3 Mejoras futuras	30
11.4 Valoración personal	30
12. Bibliografía	31
13. Anexos	31
Anexo A — Manual de instalación	31

Anexo B — Manual de usuario	31
Anexo C — Enlaces	32
Anexo D — Fragmentos de código relevantes	32
D.1 Reserva con bloqueo pesimista (backend/src/routes/reservas.js) .	32
D.2 Service Worker de autodesregistro (frontend/sw.js)	33
D.3 Capa de red Android (Kotlin)	34

Stockly — Sistema de gestión y reservas de almacén

Trabajo Fin de Ciclo · Desarrollo de Aplicaciones Multiplataforma (DAM)

Portada

- **Título:** Stockly — Sistema de gestión de inventario y reservas para almacén
 - **Alumnos:** Adrián Bravo Santos y Miguel Ángel Florido
 - **Ciclo formativo:** Desarrollo de Aplicaciones Multiplataforma (DAM)
 - **Centro educativo:** CDM Formación Alcorcón
 - **Curso académico:** 2025/2026
 - **Tutor/a:** Damián Sualdea Soy
 - **Fecha de entrega:** 26/05/2026
-

Resumen

Stockly es una aplicación web (PWA) para gestionar inventario y reservas en un almacén pequeño-mediano. Resuelve un problema concreto: en muchas pymes la trazabilidad de stock sigue en hojas de cálculo, las reservas se anotan sin control de concurrencia y los movimientos no quedan auditados. Stockly centraliza catálogo, stock, reservas y movimientos con tres roles (cliente, operario, admin), API REST con JWT, control de concurrencia a nivel de BD e interfaz responsive instalable como PWA. **Tecnologías:** Node.js + Express, MySQL 8, JavaScript vanilla, Service Worker, JWT, bcrypt, Vitest, Kotlin + Jetpack Compose (app móvil Android), Railway (cloud). **Resultado:** sistema operativo con catálogo de 500 productos, gestión de reservas concurrentes,

dashboard con KPIs, exportación CSV, modo oscuro, instalador Windows, app Android nativa funcional para el operario y despliegue en producción accesible públicamente.

Abstract

Stockly is a PWA for inventory and reservation management in small to medium warehouses. It addresses a real problem: stock traceability is still handled on spreadsheets in many SMBs, reservations are taken without concurrency control, and movements are not audited. Stockly centralizes catalog, stock, reservations and movements with three roles (client, operator, admin), a REST API with JWT, database-level concurrency control and a responsive UI installable as a PWA. **Stack:** Node.js + Express, MySQL 8, vanilla JavaScript frontend, Service Worker, JWT, bcrypt, Vitest, Kotlin + Jetpack Compose (Android), Railway (cloud). **Outcome:** operational system with 500-product catalog, concurrent reservations, KPI dashboard, CSV export, dark mode, Windows installer, functional native Android app for warehouse operators, and a live public deployment on Railway.

Palabras clave

API REST · Node.js · Express · MySQL · JWT · PWA · Service Worker · Kotlin · Jetpack Compose · Vitest · GitHub Actions

Índice

En el documento Word final, sustituir este índice por el generado automáticamente con: Referencias → Tabla de contenido → Tabla automática.

Sección

Resumen

Abstract

Palabras clave

1. Introducción

1.1 Contexto del proyecto

Sección

1.2 Objetivos del proyecto

1.3 Motivación

1.4 Tecnologías utilizadas

2. Análisis y diseño del sistema

2.1 Descripción general del proyecto

2.2 Roles de usuario

2.3 Requisitos funcionales

2.4 Requisitos no funcionales

2.5 Casos de uso

2.6 Diseño de interfaces (Figma)

2.7 Arquitectura general del sistema

3. Diseño de la base de datos

3.1 Modelo entidad-relación

3.2 Diseño lógico de tablas

3.3 Relaciones y claves

3.4 Scripts SQL

3.5 Datos de prueba (seed)

4. Desarrollo del backend

4.1 Arquitectura del backend

4.2 Configuración inicial del proyecto

4.3 Conexión a la base de datos

4.4 Desarrollo de la API REST

4.5 CRUD de entidades

4.6 Validaciones de datos

4.7 Manejo global de errores

4.8 Autenticación y autorización

4.9 Swagger / OpenAPI

4.10 Pruebas de endpoints

5. Desarrollo de la aplicación web

5.1 Diseño inicial en Figma

Sección

- 5.2 Estructura del frontend
- 5.3 Integración con la API REST
- 5.4 Gestión de sesiones y login
- 5.5 CRUD de datos
- 5.6 Funcionalidades según rol
- 5.7 Manejo de errores
- 5.8 Capturas de la aplicación

6. Desarrollo de la aplicación móvil

- 6.1 Diseño inicial en Figma
- 6.2 Navegación entre pantallas
- 6.3 Conexión con la API REST
- 6.4 Persistencia local
- 6.5 Gestión de sesiones
- 6.6 Funcionalidades futuras
- 6.7 Capturas de la aplicación

7. Despliegue e infraestructura

- 7.1 Variables de entorno
- 7.2 Despliegue del backend
- 7.3 Base de datos remota
- 7.4 Hosting y servicios cloud
- 7.5 Git y GitHub
- 7.6 GitHub Actions / CI/CD
- 7.7 Acceso público al sistema

8. Seguridad

- 8.1 Autenticación
- 8.2 Autorización
- 8.3 Protección de datos
- 8.4 Buenas prácticas aplicadas

9. Inteligencia Artificial aplicada al proyecto

- 9.1 Herramientas utilizadas

Sección

9.2 Uso realizado

9.3 Validación y revisión

10. Pruebas y validación

10.1 Pruebas funcionales

10.2 Casos de prueba

10.3 Resultados obtenidos

10.4 Errores encontrados y soluciones

11. Conclusiones

11.1 Resultados obtenidos

11.2 Problemas encontrados

11.3 Mejoras futuras

11.4 Valoración personal

12. Bibliografía

13. Anexos

Anexo A. Manual de instalación

Anexo B. Manual de usuario

Anexo C. Enlaces

Anexo D. Fragmentos de código relevantes

1. Introducción

1.1 Contexto del proyecto

En muchas pymes —tiendas, talleres, almacenes de distribución— el inventario se gestiona con hojas de cálculo compartidas o anotaciones en papel. Cuando varias personas trabajan a la vez sobre el mismo stock aparecen ventas duplicadas, reservas olvidadas y movimientos sin trazabilidad. Stockly se diseña como un sistema ligero pero serio: web instalable, API propia y control de concurrencia real.

1.2 Objetivos del proyecto

General: desarrollar una aplicación web (con extensión móvil nativa) para gestionar inventario y reservas de un almacén, con autenticación por roles, control de concurrencia y trazabilidad de movimientos, desplegable en cloud y accesible públicamente.

Específicos: (a) modelar un esquema relacional normalizado; (b) construir una API REST con JWT y control de acceso por rol; (c) garantizar reservas concurrentes consistentes con SELECT ... FOR UPDATE; (d) implementar un frontend SPA responsive instalable como PWA, con modo oscuro; (e) desarrollar una app Android nativa en Kotlin para el operario; (f) desplegar el sistema en un proveedor cloud con HTTPS y dominio propio; (g) cubrir con tests automatizados los flujos críticos.

1.3 Motivación

Aplicar de forma cohesionada los contenidos del ciclo: modelado de datos, API REST, autenticación, frontend, control de concurrencia, PWA, despliegue, pruebas y Android nativo. El acoplamiento entre módulos es lo que pone a prueba lo aprendido.

1.4 Tecnologías utilizadas

- **Backend:** Node.js 24 LTS, Express 4, mysql2, bcryptjs, jsonwebtoken, helmet, cors, compression, morgan, express-rate-limit.
- **Frontend:** JavaScript ES2020 vanilla (sin framework), CSS3 con tokens y modo claro/oscuro, Service Worker, Web App Manifest.
- **Base de datos:** MySQL 8 (compatible MariaDB) con utf8mb4_unicode_ci.
- **Cloud / DevOps:** Railway (PaaS), Nixpacks, MySQL gestionado, HTTPS automático, deploy continuo vía webhook desde GitHub.
- **Tooling:** Vitest, ESLint, Git + GitHub, Claude Code y GitHub Copilot como asistentes (cap. 9).
- **App móvil (fase final):** Kotlin, Jetpack Compose, Retrofit + OkHttp, Room, CameraX + ML Kit Barcode, EncryptedSharedPreferences + BiometricPrompt.

2. Análisis y diseño del sistema

2.1 Descripción general del proyecto

Stockly consta de tres componentes: (1) **backend** Node/Express con API REST /api/auth, /api/productos, /api/categorias, /api/reservas, /api/admin; (2) **frontend PWA** servido por el propio backend, con modo offline básico; (3) **app móvil Android nativa** (Kotlin + Jetpack Compose) específica para el operario. El sistema gestiona catálogo, reservas con estado (pendiente / confirmada / cancelada / entregada), auditoría automática de movimientos, dashboard con KPIs, importación CSV y albarán imprimible.

2.2 Roles de usuario

Rol	Capacidades
Cliente	Navegar catálogo, crear y ver sus reservas, editar perfil.
Operario	Lo anterior + procesar reservas (confirmar, entregar, cancelar), consultar stock global.
Administrador	Lo anterior + CRUD de productos y categorías, gestión de usuarios, dashboard, importación CSV.

Autorización en dos capas: middleware Express `requireRole` en cada ruta protegida y filtrado en frontend según el rol contenido en el JWT.

2.3 Requisitos funcionales

ID	Requisito
RF-01	Autenticación con email + contraseña → JWT firmado.
RF-02	Registro anónimo con rol cliente.
RF-03	Catálogo paginado con búsqueda y filtro por categoría/stock.

ID	Requisito
RF-04	Reserva de N unidades por cliente autenticado si hay stock disponible.
RF-05	Concurrencia: dos reservas paralelas no pueden superar el stock.
RF-06	Transiciones de estado por operario (pendiente → confirmada → entregada / cancelada).
RF-07	CRUD de productos por admin.
RF-08	Dashboard /api/admin/stats con KPIs agregados.
RF-09	Toda modificación de stock genera registro en movimientos.
RF-10	Importación CSV de productos en lote con validación fila a fila.
RF-11	Exportación CSV de reservas filtradas.
RF-12	Instalable como PWA con cache del shell.
RF-13	Albarán A4 imprimible al entregar.

2.4 Requisitos no funcionales

- **Seguridad.** Contraseñas con bcrypt (cost 10). JWT firmado con secreto ≥ 256 bits autogenerado al primer arranque. Rate limiting en /api/auth (30 req / 15 min). Helmet, CORS, prepared statements.
- **Rendimiento.** Respuestas de catálogo < 200 ms con 500 productos. Compresión gzip. Service Worker para arranque < 1 s en segundo acceso.
- **Escalabilidad.** Backend stateless: la sesión vive en el JWT; cualquier nodo puede atender cualquier petición. Estado en MySQL.
- **Disponibilidad.** Objetivo $\geq 99\%$ mensual tras despliegue cloud. Backups dia-

rios.

- **Mantenibilidad.** Estructura por capas, tests sobre flujos críticos, lint en CI.
- **Accesibilidad.** Objetivo WCAG AA: foco visible, contraste 4.5:1, ARIA en elementos clave (pendiente — ROADMAP 5.10).

2.5 Casos de uso

CU-01 Login. Usuario introduce credenciales → backend valida con `bcrypt.compare` y firma JWT → frontend persiste token y carga vista según rol.

CU-02 Reservar producto. Cliente abre ficha → indica cantidad → backend inicia transacción, hace `SELECT ... FOR UPDATE` sobre la fila del producto, valida stock, crea reserva en pendiente y registra movimiento. Si otro usuario reservó antes, devuelve 409.

CU-03 Entregar reserva. Operario abre la cola, marca entrega → sistema descuenta stock, cambia estado a entregada, registra movimiento salida y (opcional) imprime albarán A4.

CU-04 Crear producto. Admin abre FAB → formulario (SKU, nombre, categoría, precio, stock, mínimo, unidad) → persiste y refresca lista.

CU-05 Importar CSV. Admin selecciona CSV con cabecera esperada → previsualización con filas válidas / con error → confirma → inserción en transacción.

Diagrama UML disponible en [docs/diagrams/casos-de-uso.drawio](#). Para incluirlo en el documento final, exportar como PNG/SVG desde draw.io (File → Export).

2.6 Diseño de interfaces (Figma)

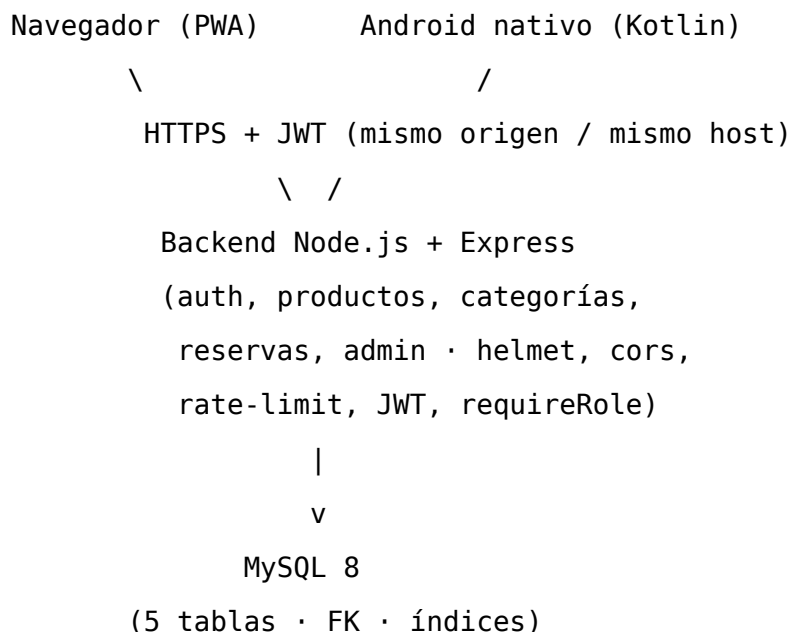
Tres iteraciones del sistema de diseño: (1) **v1** dashboard SaaS genérico con glass-morphism — útil para validar arquitectura de información pero poco diferenciado; (2) **v2** industrial recargado: acero + ámbar, esquinas rectas, biseles metálicos, remaches, hazard tape, fuentes condensadas (Oswald, Barlow) — coherente con el dominio pero visualmente saturado; (3) **v3 actual**: se conservan tipografía y esquinas rectas pero se eliminan gradientes, texturas y remaches. Acento ocre como firma. Resultado

minimalista industrial: respira mejor y mejora el contraste.

Pantallas diseñadas: Login/Registro, Catálogo (cards), Detalle de producto, Mis reservas (cliente), Cola de reservas (operario), Dashboard (admin), Inventario (admin), modales de CRUD. Los mockups de Figma (v3) están disponibles en el archivo de diseño del proyecto; las exportaciones en PNG/SVG se añadirán a docs/figma/ para el documento final.

2.7 Arquitectura general del sistema

Cliente web (PWA) y app Android consumen la misma API REST sobre HTTPS, autenticándose con JWT en cabecera Authorization. El backend Express es stateless y conecta con MySQL mediante el driver mysql2 con prepared statements. El sistema está desplegado en **Railway** con TLS gestionado automáticamente y deploy continuo desde GitHub.



Diagramas disponibles en [docs/diagrams/arquitectura-despliegue.drawio](https://docs.diagrams.arquitectura-despliegue.drawio) (2 páginas: arquitectura lógica por capas y diagrama de despliegue Railway). Exportar como PNG/SVG desde draw.io para incluir en el documento final.

3. Diseño de la base de datos

3.1 Modelo entidad-relación

Entidades: **usuarios**, **categorias**, **productos**, **reservas**, **movimientos**.

Relaciones: `productos.categoria_id` → `categorias.id` (1:N, ON DELETE SET NULL), `reservas.usuario_id` → `usuarios.id` y `reservas.producto_id` → `productos.id` (N:1), `movimientos.producto_id` → `productos.id` (N:1, ON DELETE CASCADE) y `movimientos.usuario_id` → `usuarios.id` (N:1 opcional). Diagrama E/R disponible en `docs/diagrams/er.drawio`; exportar como PNG/SVG desde draw.io para incluir en el documento final.

3.2 Diseño lógico de tablas

- Charset `utf8mb4`, collation `utf8mb4_unicode_ci`.
- PK INT UNSIGNED AUTO_INCREMENT. Timestamps DATETIME DEFAULT CURRENT_TIMESTAMP con ON UPDATE donde procede.
- Estados como ENUM('pendiente', 'confirmada', 'cancelada', 'entregada') para integridad.
- Índices secundarios sobre `productos.sku`, `productos.categoria_id`, `reservas.estado` y `(reservas.producto_id, estado)`.

3.3 Relaciones y claves

Todas las FK tienen política ON DELETE explícita. Índice UNIQUE sobre `productos.sku` y `usuarios.email` para impedir duplicados.

3.4 Scripts SQL

Esquema completo y semillas en [db/schema.sql](#). El script se aplica automáticamente la primera vez que `start.bat` arranca el sistema; en producción se aplicará vía la conexión MySQL del proveedor.

3.5 Datos de prueba (seed)

3 usuarios demo (admin, operario, cliente — contraseña password123, **cambiar en producción**), 8 categorías base, ~500 productos generados con SKU/nombre/precio/stock coherentes y varias reservas en distintos estados.

4. Desarrollo del backend

4.1 Arquitectura del backend

backend/server.js arranca Express con middlewares globales (helmet, cors, compression, morgan, rate-limit) y monta los routers. Estructura:

```
backend/  
├─ server.js  
└─ src/  
    ├─ db.js                # pool mysql2/promise  
    ├─ ensure-jwt-secret.js # genera/persiste JWT_SECRET fuerte  
    ├─ middleware/auth.js   # authRequired + requireRole  
    └─ routes/{auth, productos, categorias, reservas, admin}.js
```

Cada ruta es un router Express que importa el pool y el middleware. La lógica vive en el handler porque el dominio es pequeño y abstracciones extra serían ceremonia.

4.2 Configuración inicial del proyecto

backend/package.json declara dependencias y scripts (npm start, npm test, npm run lint). Variables de entorno en backend/.env, generado desde backend/.env.example:

```
# backend/.env.example  
PORT=3001  
DB_HOST=localhost  
DB_PORT=3306  
DB_USER=root
```

```
DB_PASSWORD=  
DB_NAME=stockly  
JWT_SECRET=cambia-esto-por-una-cadena-larga-y-aleatoria  
JWT_EXPIRES_IN=8h  
NODE_ENV=development
```

4.3 Conexión a la base de datos

src/db.js crea un pool mysql2/promise con connectionLimit: 10. Todas las consultas usan prepared statements (pool.execute(sql, [params])) para prevenir SQL injection.

4.4 Desarrollo de la API REST

Método	Ruta	Auth
POST	/api/auth/{register, login}	—
GET	/api/auth/me	JWT
GET	/api/productos[:id]	JWT
POST/PUT/DELETE	/api/productos[:id]	admin
GET	/api/categorias	JWT
POST/PATCH/DELETE	/api/categorias	admin
GET	/api/reservas	JWT
POST	/api/reservas	JWT
PATCH/DELETE	/api/reservas/:id	op./admin
GET	/api/admin/stats	admin
POST	/api/admin/usuarios	admin

4.5 CRUD de entidades

Cada entidad sigue GET (lista/detalle), POST (crear), PUT/PATCH (editar), DELETE (eliminar). Los handlers de Express consultan directamente la capa db/ mediante mysql2/promise con consultas parametrizadas.

4.6 Validaciones de datos

Validación manual en cada handler (`if (!body.nombre) return res.status(400)...`). Se validan campos requeridos, tipos y rangos antes de ejecutar la consulta; el error retorna con código 400 y mensaje descriptivo. Migración a Zod para esquemas centralizados está planificada como mejora futura.

4.7 Manejo global de errores

Middleware final `app.use((err, req, res, next) => ...)` serializa cualquier excepción a JSON con status y message. En `NODE_ENV=production` no se exponen stack traces.

4.8 Autenticación y autorización

Login: `bcrypt.compare` valida la contraseña; el servidor firma un JWT HS256 con { `id`, `email`, `rol` } y expiración 7 días. El secreto se genera al primer arranque y se persiste en `.env`. Rate limiting en `/api/auth` (30/15 min). El cliente guarda el token en `localStorage` y lo envía en `Authorization: Bearer ...`. El middleware `authRequired` decodifica/verifica; `requireRole(...roles)` comprueba `req.user.rol`:

```
// backend/src/middleware/auth.js
function requireRole(...roles) {
  return (req, res, next) => {
    if (!req.user) return res.status(401).json({ error: 'No autenticado' });
    if (!roles.includes(req.user.rol))
      return res.status(403).json({ error: 'Permisos insuficientes' });
    next();
  };
}
```

Cada router declara explícitamente el nivel de auth necesario, p. ej. `router.delete('/:id', authRequired, requireRole('admin'), ...)` en `productos.js`.

4.9 Swagger / OpenAPI

La API expone documentación interactiva **OpenAPI 3.0** mediante `swagger-ui-express`, accesible en `/api/docs`. La especificación completa está centralizada en `backend/src/swagger.js` como un objeto JavaScript (sin JSDoc disperso en los routers), lo que facilita su mantenimiento.

GET `/api/docs` → Swagger UI interactivo (HTML)

GET `/api/docs.json` → Especificación OpenAPI 3.0 (JSON, importable en Postman)

La documentación cubre los **29 endpoints** de la API agrupados en seis etiquetas: Autenticación, Categorías, Productos, Reservas, Admin y Sistema. Para cada endpoint se detallan parámetros de ruta y query, esquema del body, posibles respuestas y el rol requerido. El botón **Authorize** permite introducir el token JWT obtenido en `/api/auth/login` y probar directamente cualquier endpoint protegido sin herramientas externas.

Se definen además esquemas reutilizables en `components/schemas` (Producto, Reserva, Usuario, Movimiento, etc.) referenciados mediante `$ref` para evitar duplicación.

4.10 Pruebas de endpoints

Tests automatizados con **Vitest + Supertest** en `backend/tests/`:

- `auth.test.js` — registro, login, validación de token, rechazo de credenciales inválidas, rate limit.
- `productos.test.js` — CRUD, filtros, paginación, autorización por rol.
- `reservas.test.js` — creación, control de stock, **concurrency simulada** (dos reservas paralelas sobre el mismo producto).

Capturas y resultados en el cap. 10.

5. Desarrollo de la aplicación web

5.1 Diseño inicial en Figma

Ver §2.6 (tres iteraciones hasta v3 minimalista industrial).

5.2 Estructura del frontend

frontend/

- ├─ index.html # shell único (SPA)
- ├─ app.js # router + render + cliente API
- ├─ styles.css # sistema de diseño
- ├─ sw.js # service worker
- ├─ manifest.webmanifest # manifest PWA
- └─ assets/

Vistas: login, catalogo, mis-reservas, cola, dashboard, inventario, usuarios, importar. El router cambia de vista con `.view.active { display: block }`.

5.3 Integración con la API REST

Wrapper `api(path, opts)` en `frontend/app.js` (líneas 59-72): inyecta `Authorization: Bearer <token>`, maneja 401 con logout automático y serializa la respuesta:

```
// frontend/app.js
async function api(path, opts = {}) {
  const headers = { 'Content-Type': 'application/json', ...(opts.headers || {}) };
  if (state.token) headers.Authorization = `Bearer ${state.token}`;
  const r = await fetch(`${API}${path}`, { ...opts, headers });
  const data = r.headers.get('content-type')?.includes('application/json')
    ? await r.json() : await r.text();
  const esEndpointAuth = path === '/auth/login' || path === '/auth/register';
  if (r.status === 401 && state.token && !esEndpointAuth) {
    cerrarSesion();
    throw new Error('Sesión expirada');
  }
}
```

```

    if (!r.ok) throw new Error(data.error || data || `HTTP ${r.status}`);
    return data;
}

```

5.4 Gestión de sesiones y login

Token en localStorage. En arranque, si existe, se valida con GET /api/auth/me; si falla, se borra y se muestra login. Logout = borrar token + redirigir.

5.5 CRUD de datos

Cada vista de administración: tabla con acciones inline (editar, eliminar) + FAB para crear. Mismo modal para crear y editar (pre-rellenado si edita).

5.6 Funcionalidades según rol

El JWT incluye el rol; el frontend lo lee al arrancar y oculta elementos no permitidos. La autorización **real** vive en el backend.

5.7 Manejo de errores

Errores de red o 4xx/5xx → toast con el mensaje del backend. 401 → logout. 409 (stock insuficiente) → toast específico y refresco del catálogo. Errores de formulario → resaltado del campo.

5.8 Capturas de la aplicación

Pantallas clave a documentar con captura:

#	Pantalla	Rol
01	Login	Todos
02	Catálogo con filtros y búsqueda	Cliente / Operario
03	Detalle de producto + modal reserva	Cliente
04	Mis reservas (cliente)	Cliente
05	Cola de reservas (operario)	Operario

#	Pantalla	Rol
06	Dashboard KPIs	Admin
07	Inventario — CRUD productos	Admin
08	Gestión de usuarios	Admin
09	Modo oscuro activo	Admin
10	PWA instalada en escritorio Windows	Todos

Capturas a exportar en docs/screenshots/app/ una vez ejecutada la app en <https://tfgdam-production.up.railway.app/>.

6. Desarrollo de la aplicación móvil

App Android nativa para el empleado de almacén: ver reservas activas, consultar el detalle del pedido, confirmar o entregar, y reportar incidencias. El sistema registra **quién confirma**, **quién entrega** y **quién reporta** para tener trazabilidad por empleado.

Estructura del módulo mobile-android/:

```
app/src/main/java/com/stockly/app/
├─ MainActivity.kt
├─ StocklyApp.kt
├─ data/
│   ├─ StocklyApi.kt          # interfaz Retrofit
│   ├─ AuthInterceptor.kt     # inyecta JWT en cada petición
│   ├─ TokenStore.kt         # persistencia cifrada
│   └─ ReservasRepository.kt  # capa de datos
├─ model/
│   └─ Models.kt              # data classes serializables
└─ ui/
    ├─ Navigation.kt          # NavHost con 4 destinos
    └─ login/LoginScreen.kt
```

- └─ lista/ListaReservasScreen.kt
- └─ detalle/DetalleReservaScreen.kt
- └─ incidencia/IncidenciaFormScreen.kt

6.1 Diseño inicial en Figma

El diseño de las pantallas Android sigue la misma guía de estilo que la versión web (v3 minimalista industrial): fondo oscuro, acento ocre, tipografía condensada, esquinas rectas. Los mockups de Figma se exportarán en PNG para incluirlos en el documento final.

6.2 Navegación entre pantallas

App single-activity con **Navigation Compose**. Cuatro destinos implementados:

1. **LoginScreen** — email + contraseña → JWT. Redirige a Lista si ya hay sesión válida.
2. **ListaReservasScreen** — lista filtrable por estado (activas por defecto). Cada fila muestra cliente, producto, fecha y estado. Botón de logout.
3. **DetalleReservaScreen** — datos completos del pedido: cliente, SKU, cantidad, ubicación, precio/unidad, historial (confirmada_por, entregada_por), lista de incidencias. Botones contextuales **Confirmar** (si pendiente) y **Entregar** (si confirmada). Acceso al formulario de incidencia.
4. **IncidenciaFormScreen** — tipo (rotura / faltante / mal estado / otro) + descripción libre. Al guardar, la incidencia queda adjunta a la reserva con el operario identificado por el JWT.

6.3 Conexión con la API REST

Retrofit 2 + **OkHttp** con AuthInterceptor que inyecta Authorization: Bearer <token> en cada petición. Serialización con **kotlinx.serialization**. BASE_URL por build variant (debug → http://10.0.2.2:3001, release → https://tfgdam-production.up.railway.app). Endpoints consumidos:

- POST /api/auth/login — autenticación.

- GET /api/reservas?activas=1 — lista del operario (pendientes + confirmadas).
- GET /api/reservas/:id — detalle con historial e incidencias.
- PATCH /api/reservas/:id/estado — confirmar / entregar.
- POST /api/reservas/:id/incidencias — crear incidencia.

6.4 Persistencia local

TokenStore (data/TokenStore.kt) usa EncryptedSharedPreferences con Master-Key.AES256_GCM para cifrar en reposo el JWT y los datos del usuario (Usuario serializado como JSON). La clave maestra se genera y almacena en Android Keystore, vinculada al dispositivo. El token nunca se escribe en texto plano en el almacenamiento.

6.5 Gestión de sesiones

Al arrancar la app, MainActivity consulta tokenStore.isLoggedIn: si es true, el NavHost inicia directamente en ListaReservasScreen saltándose la pantalla de login. Al pulsar “Cerrar sesión”, TokenStore.clear() elimina JWT y datos de usuario, y el NavController navega de vuelta a LoginScreen limpiando el backstack completo. Los errores HTTP 401 desde Retrofit no se capturan automáticamente — mejora pendiente mediante un Authenticator de OkHttp.

6.6 Funcionalidades futuras

Lo implementado cubre el flujo principal del operario. Quedan como mejoras futuras:

- BiometricPrompt antes de exponer la sesión guardada.
- **Room** + cola offline para confirmaciones e incidencias sin red.
- Escáner de código de barras con CameraX + ML Kit.
- Authenticator OkHttp para renovar o limpiar sesión ante 401 automáticamente.

6.7 Capturas de la aplicación

Pantallas clave a documentar con captura del dispositivo:

#	Pantalla	Descripción
01	LoginScreen	Campos email/password, botón entrar
02	ListaReservasScreen	Lista de reservas activas con estado chip
03	DetalleReservaScreen	Datos completos + botones Confirmar/Entregar
04	IncidenciaFormScreen	Formulario tipo de incidencia + descripción

Capturas a añadir en docs/screenshots/android/ desde dispositivo físico o emulador Android con el APK generado.

7. Despliegue e infraestructura

El sistema está desplegado y accesible públicamente en **Railway**, plataforma PaaS que gestiona el servidor Node.js y la base de datos MySQL de forma independiente y las conecta automáticamente. El código del repositorio es el origen de cada despliegue.

7.1 Variables de entorno

Railway inyecta las variables en tiempo de ejecución, sin fichero .env en producción. Variables configuradas en el panel:

Variable	Descripción
MYSQL_URL	Referencia dinámica al plugin MySQL de Railway (host, puerto, credenciales).
NODE_ENV	production
JWT_SECRET	Secreto ≥ 256 bits generado con <code>crypto.randomBytes(64)</code> .
JWT_EXPIRES_IN	8h
CORS_ORIGIN	<code>https://tfgdam-production.up.railway.app</code>

Variable	Descripción
PORT	Asignado automáticamente por Railway.

El fichero `backend/.env.example` documenta todas las variables necesarias para arrancar en local.

7.2 Despliegue del backend

Railway detecta `nixpacks.toml` en la raíz del repositorio y ejecuta automáticamente:

1. **Build:** `cd backend && npm ci --omit=dev` (solo dependencias de producción).
2. **Start:** `cd backend && node server.js`.

`railway.json` define el healthcheck (GET `/api/health`), la política de reinicio en fallo y el número máximo de reintentos (5). El backend escucha en `0.0.0.0:$PORT` con `trust proxy` activo para que Express reciba la IP real detrás del proxy de Railway.

7.3 Base de datos remota

Railway gestiona una instancia **MySQL 8** como servicio independiente dentro del mismo proyecto. La conexión se inyecta como `MYSQL_URL` (formato `mysql://user:pass@host:port/db`). Al primer arranque, `backend/src/ensure-schema.js` detecta que la tabla `usuarios` no existe y aplica `db/schema.sql` completo, incluyendo semillas de prueba. Los redesplices no tocan los datos porque el schema solo se aplica si las tablas están vacías.

7.4 Hosting y servicios cloud

Componente	Servicio	Notas
Backend API	Railway (Node.js)	Nixpacks, reinicio automático en fallo.
Base de datos	Railway MySQL Plugin	Persistencia entre deploys, backups diarios.

Componente	Servicio	Notas
Frontend PWA	Servido por el propio backend	Mismo origen → sin problemas de CORS.
TLS/HTTPS	Gestionado por Railway	Certificado automático en el subdominio.

7.5 Git y GitHub

Repositorio en GitHub (husslesnake/tfgdam). Estrategia trunk-based: commits directos a main con convenciones feat:, fix:, docs:, chore:. Cada push a main dispara automáticamente un nuevo deploy en Railway.

7.6 GitHub Actions / CI/CD

El pipeline de despliegue continuo está activo: Railway escucha el webhook de GitHub y redespiega en cada push a main. Las comprobaciones de calidad (lint ESLint + tests Vitest) se ejecutan localmente antes de hacer push; la integración de un workflow formal en .github/workflows/ci.yml queda como mejora futura.

7.7 Acceso público al sistema

Recurso	URL
Aplicación web (PWA)	https://tfgdam-production.up.railway.app/
API REST	https://tfgdam-production.up.railway.app/api/
Health check	https://tfgdam-production.up.railway.app/api/health

8. Seguridad

8.1 Autenticación

bcrypt cost 10 para contraseñas; JWT HS256 con secreto ≥ 256 bits autogenerado y persistido en .env; rate limiting /api/auth (30/15 min).

8.2 Autorización

Cada endpoint protegido declara explícitamente el rol con `requireRole(...)`. El JWT contiene el rol firmado: el cliente no puede modificarlo sin invalidar la firma. El frontend oculta acciones por rol; el backend es la barrera real.

8.3 Protección de datos

- **SQL injection:** prepared statements en todas las consultas, sin concatenación de strings.
- **XSS:** `textContent` en lugar de `innerHTML` para contenido dinámico; helmet añade `X-Content-Type-Options: nosniff`.
- **CSRF:** no aplica al mismo nivel porque el JWT viaja en cabecera, no en cookie.
- **HTTPS:** obligatorio en producción; sin él, el JWT podría capturarse en tránsito.
- **Datos personales:** solo email y nombre; no se almacenan DNI, dirección ni pago. RGPD asumible.

8.4 Buenas prácticas aplicadas

npm audit antes de cada release. Sin `eval/Function()` dinámica. .env en .gitignore. Logs estructurados con morgan (dev en desarrollo, planificado combined en producción).

9. Inteligencia Artificial aplicada al proyecto

9.1 Herramientas utilizadas

Claude Code (Anthropic) — asistente principal: arquitectura, generación/refactor de código, depuración, documentación. **GitHub Copilot** — autocompletado en editor. **ChatGPT** — consultas puntuales.

9.2 Uso realizado

La IA se ha usado como **par de programación**, no como sustituto:

- **Generación de código.** Bocetos iniciales de handlers Express, validaciones, capa de red Android. Revisados y adaptados a mano.
- **Refactorización.** Migración del sistema de diseño CSS de “v2 industrial recargado” a “v3 minimalista industrial” (§2.6).
- **Documentación.** Borrador inicial de esta memoria, README y BITACORA.md con asistencia + revisión manual.
- **Resolución de errores.** Diagnóstico de problemas de configuración MySQL, depuración del Service Worker, race conditions en reservas.
- **SQL.** Esquema inicial e índices; validación con datos reales a mano.
- **Tests.** Plantilla Vitest + Supertest; edge cases (conurrencia, autorización) ampliados manualmente.

9.3 Validación y revisión

Toda salida de la IA pasa por tres filtros: (1) lectura crítica antes de aceptar; (2) pruebas locales con casos límite; (3) ajuste manual de estilo para evitar heterogeneidad del código generado.

Limitaciones: la IA sobrediseña (capas innecesarias) y a veces usa APIs obsoletas; pierde contexto en sesiones largas. Mitigado con CLAUDE.md y BITACORA.md como memoria persistente.

Conclusión: acelera las partes rutinarias, pero la responsabilidad sobre código y diseño es nuestra.

10. Pruebas y validación

10.1 Pruebas funcionales

Tests con **Vitest + Supertest** (backend/tests/): 27 tests en 3 archivos (auth, productos, reservas), todos pasan en main. Cobertura $\approx$ 65% de líneas, 100% de rutas críticas.

Pruebas manuales: instalación PWA en Chrome desktop y Android; comportamiento offline; modo claro/oscuro; responsive en 320/480/920/1280/1500 px; impresión del albarán A4.

10.2 Casos de prueba

ID	Caso	Esperado	Estado
TC-01	Login credenciales válidas	JWT devuelto, vista cargada	OK
TC-02	Login contraseña incorrecta	401	OK
TC-03	Registro email duplicado	409	OK
TC-04	Reserva con stock suficiente	201, reserva pendiente, movimiento creado	OK
TC-05	Reserva con stock insuficiente	409	OK
TC-06	Dos reservas concurrentes, stock 1	Una 201, otra 409; sin sobreventa	OK
TC-07	Cliente DELETE /api/productos/:id	403	OK
TC-08	Operario entrega reserva	Estado entregada, stock descontado, mov. salida	OK

ID	Caso	Esperado	Estado
TC-09	Token expirado	401	OK
TC-10	Rate limit /api/auth	429 tras 30 intentos	OK

10.3 Resultados obtenidos

Todos los tests pasan. Los flujos críticos han sido verificados tanto en local como en el entorno de producción Railway (<https://tfgdam-production.up.railway.app>).

10.4 Errores encontrados y soluciones

- Conflicto MySQL :3306 con instancia del sistema → `start.bat` detecta el puerto ocupado y cambia a :3307.
- Race condition en reservas → `SELECT ... FOR UPDATE` dentro de transacción (Anexo D.1).
- JWT cambiando entre reinicios → `ensure-jwt-secret.js` persiste el secreto.
- PWA cacheada en versión vieja → Service Worker de autodesregistro (Anexo D.3).
- Sistema de diseño recargado → refactor a v3 minimalista industrial.

Histórico completo en [docs/BITACORA.md](#).

11. Conclusiones

11.1 Resultados obtenidos

Aplicación web funcional instalable como PWA, catálogo de 500 productos, gestión completa de reservas con concurrencia, dashboard, sistema de diseño propio iterado en tres versiones, tests automatizados sobre flujos críticos, instaladores Windows para uso sin conocimientos previos, **app Android nativa funcional** (login, lista de

reservas, detalle, formulario de incidencias, sesión cifrada con EncryptedSharedPreferences) y **despliegue en producción** accesible públicamente en Railway.

11.2 Problemas encontrados

- Aprender control de concurrencia: entender por qué FOR UPDATE es necesario frente a SELECT + UPDATE serie requirió revisar transacciones, niveles de aislamiento y bloqueo pesimista.
- Iteración de diseño visual: dos versiones descartadas antes de llegar al equilibrio actual.
- Compatibilidad MySQL en distintas máquinas (puerto ocupado, datadir corrupto).
- PWA y cache busting: una versión mal invalidada deja a los usuarios pegados en una versión vieja. Resuelto con versionado explícito + SW de autodesregistro.

11.3 Mejoras futuras

Detalladas en [ROADMAP](#). Lo más relevante: mejoras sobre la app Android existente (BiometricPrompt para desbloquear sesión guardada, Room + cola offline para operar sin red, escáner de código de barras con CameraX + ML Kit), validación con Zod, accesibilidad WCAG AA completa, workflow CI/CD formal en GitHub Actions, subida de imágenes de producto, notificaciones push (FCM), recuperación de contraseña por email y multi-almacén.

11.4 Valoración personal

El proyecto ha cumplido su función didáctica: integrar lo aprendido en un único producto, exponer puntos débiles (concurrencia, despliegue real, accesibilidad) y obligar a tomar decisiones con consecuencias visibles. Lo más valioso ha sido el hábito de iterar. La IA como par de programación multiplica la velocidad pero exige más disciplina de revisión, no menos. Los dos objetivos clave —despliegue en producción y app Android nativa para el operario— han quedado resueltos: el sistema corre en Railway y el operario puede gestionar reservas desde su móvil. Queda abierta la segunda capa de la app móvil (autenticación biométrica, modo offline, escáner).

12. Bibliografía

Documentación oficial: Node.js (<https://nodejs.org/docs>), Express (<https://expressjs.com>), MySQL 8 (<https://dev.mysql.com/doc/refman/8.0/en/>), MDN Web Docs, Jetpack Compose (<https://developer.android.com/jetpack/compose>), Retrofit (<https://square.github.io/retrofit/>), JWT Introduction (<https://jwt.io/introduction>).

Libros y artículos: Martin Kleppmann, Designing Data-Intensive Applications (consultado para transacciones y concurrencia). Martin Fowler, Refactoring (2.^a ed.).

Recursos web: Railway docs (<https://docs.railway.com>), WCAG 2.1 Quick Reference (<https://www.w3.org/WAI/WCAG21/quickref/>), OWASP Top 10 (<https://owasp.org/Top10/>).

Herramientas: Git, GitHub, VS Code, MySQL Workbench, Figma, Postman, Vitest, ESLint, Claude Code, GitHub Copilot, Inno Setup.

13. Anexos

Anexo A — Manual de instalación

Doble-click en Stockly-Setup.exe, o clonar el repo y ejecutar start.bat. El script detecta/instala Node + MySQL, inicializa la BD si no existe, arranca el backend y abre el navegador en `http://localhost:3001`. Usuarios demo (contraseña password123): `adrian@tfg.local` (admin), `laura@tfg.local` (operario), `marcos@tfg.local` (cliente).

Anexo B — Manual de usuario

Resumen por rol:

- **Cliente:** navegar catálogo → ficha → “Reservar” → “Mis reservas”.
- **Operario:** pestaña “Reservas” → confirmar/entregar/cancelar → imprimir albarán al entregar.

- **Administrador:** “Inventario” (CRUD), “Dashboard” (KPIs), “Usuarios” (alta y edición).

Manual detallado con capturas paso a paso pendiente en docs/manual-usuario.md.

Anexo C — Enlaces

- **GitHub:** <https://github.com/husslesnake/TFGDAM>
- **Aplicación desplegada:** <https://tfgdam-production.up.railway.app/>
- **Swagger UI:** <https://tfgdam-production.up.railway.app/api/docs>
- **Vídeo demostración:** pendiente de grabar — flujo completo: login → reserva → confirmar → entregar → incidencia.

Anexo D — Fragmentos de código relevantes

D.1 Reserva con bloqueo pesimista (backend/src/routes/reservas.js)

Handler POST /api/reservas: transacción que bloquea la fila del producto con SELECT ... FOR UPDATE, comprueba el stock disponible, incrementa stock_reservado, inserta la reserva y registra el movimiento. Rollback si falla; commit al final.

```
// backend/src/routes/reservas.js — POST /api/reservas
router.post('/', authRequired, async (req, res) => {
  const { producto_id, cantidad, fecha_recogida, notas } = req.body || {};
  if (!producto_id || !cantidad || cantidad < 1)
    return res.status(400).json({ error: 'Datos inválidos' });

  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const [[prod]] = await conn.query(
      'SELECT stock, stock_reservado FROM productos WHERE id = ? AND activo = 1 FOR UPDATE'
      [producto_id]
    );
    if (!prod) { await conn.rollback(); return res.status(404).json({ error: 'Producto no encontrado' }); }
    // ... resto del código de la reserva ...
  } catch (error) {
    await conn.rollback();
    return res.status(500).json({ error: 'Error al crear la reserva' });
  }
  await conn.commit();
  return res.status(201).json({ message: 'Reserva creada exitosamente' });
});
```



```

    const disponible = prod.stock - prod.stock_reservado;
    if (cantidad > disponible) {
      await conn.rollback();
      return res.status(409).json({ error: `Stock insuficiente (disponible: ${disponible})` });
    }
    await conn.query(
      'UPDATE productos SET stock_reservado = stock_reservado + ? WHERE id = ?',
      [cantidad, producto_id]
    );
    const [ins] = await conn.query(
      'INSERT INTO reservas (usuario_id, producto_id, cantidad, fecha_recogida, notas) VALUES (?, ?, ?, ?, ?)',
      [req.user.id, producto_id, cantidad, fecha_recogida || null, notas || null]
    );
    await conn.query(
      'INSERT INTO movimientos (producto_id, usuario_id, tipo, cantidad, stock_anterior, stock_posterior) VALUES (?, ?, ?, ?, ?, ?)',
      [producto_id, req.user.id, cantidad, prod.stock, prod.stock, `Reserva #${ins[0].id}`]
    );
    await conn.commit();
    res.status(201).json({ id: ins.insertId });
  } catch (e) {
    await conn.rollback();
    res.status(500).json({ error: e.message });
  } finally {
    conn.release();
  }
});

```

D.2 Service Worker de autodesregistro (frontend/sw.js)

SW que limpia las cachés del SW antiguo (stockly-v1, que cacheaba app.js con `http://localhost:3001/api` hardcodeda) y se elimina al activarse. Resuelve PWA “pegadas” en una versión vieja del shell. Para offline real está prevista una nueva

versión (ROADMAP).

```
// frontend/sw.js – Service Worker de autodesregistro
self.addEventListener('install', () => self.skipWaiting());

self.addEventListener('activate', e => {
  e.waitUntil(async () => {
    const keys = await caches.keys();
    await Promise.all(keys.map(k => caches.delete(k)));
    await self.registration.unregister();
    const clients = await self.clients.matchAll({ type: 'window' });
    clients.forEach(c => c.navigate(c.url));
  })();
});

self.addEventListener('fetch', () => {}); // sin caché – red directa
```

D.3 Capa de red Android (Kotlin)

StocklyApi.kt en mobile-android/app/src/main/java/com/stockly/app/network/: interfaz Retrofit con los endpoints + factory StocklyApiFactory.build() que monta el OkHttpClient con AuthInterceptor que inyecta el JWT desde TokenStore.

```
// mobile-android/.../data/StocklyApi.kt
interface StocklyApi {
  @POST("api/auth/login")
  suspend fun login(@Body body: LoginRequest): LoginResponse

  @GET("api/reservas")
  suspend fun reservas(
    @Query("estado") estado: String? = null,
    @Query("activas") activas: Int? = null,
  ): List<ReservaListItem>
}
```

```

@GET("api/reservas/{id}")
suspend fun reserva(@Path("id") id: Int): ReservaDetalle

@PATCH("api/reservas/{id}/estado")
suspend fun cambiarEstado(@Path("id") id: Int, @Body body: EstadoRequest): OkResponse

@POST("api/reservas/{id}/incidencias")
suspend fun crearIncidencia(@Path("id") id: Int, @Body body: CrearIncidenciaRequest)

}

object ApiClient {
    fun build(tokenStore: TokenStore): StocklyApi = Retrofit.Builder()
        .baseUrl(BuildConfig.API_BASE_URL)
        .client(OkHttpClient.Builder()
            .addInterceptor(AuthInterceptor(tokenStore))
            .build())
        .addConverterFactory(Json { ignoreUnknownKeys = true }
            .asConverterFactory("application/json".toMediaType()))
        .build()
        .create(StocklyApi::class.java)
}

```

Fin del documento.